

Clinic Appointment Booking System

Full-stack web application to manage doctors, patients, holidays, schedules and audit records. TypeScript monorepo: Node/Express backend, React/Vite frontend, MongoDB with Mongoose.

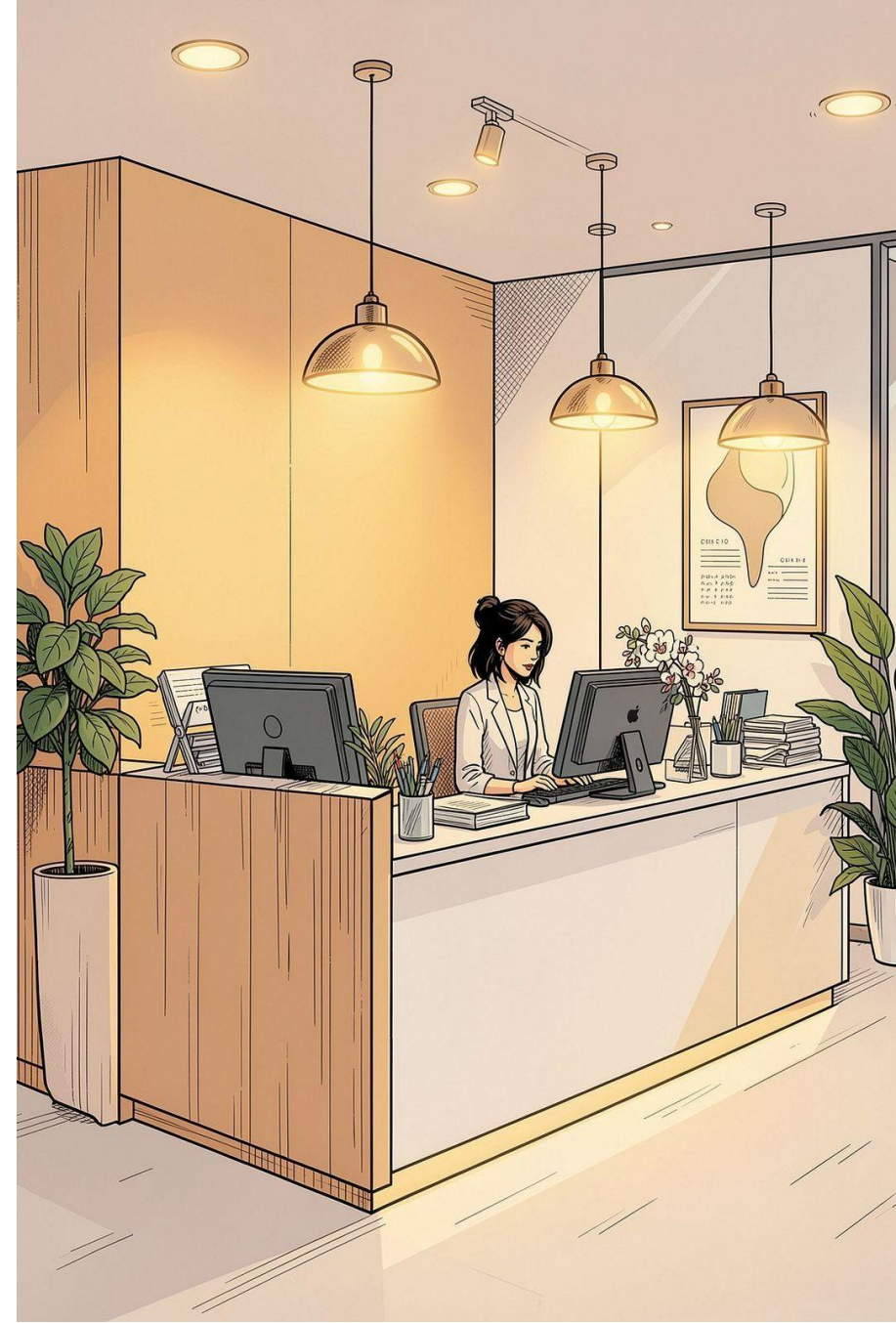
571 RYAN PIUS WILLIAM

572 SHELAVLE SAYALI MAHESH

573 SHETYE MANASI MADHUKAR

574 SINGH JATIN ANIL

575 SONKAR SURAJ VIRENDRA



Executive Summary



What it does

Browse doctors, view availability, book appointments; staff manage records and holidays.



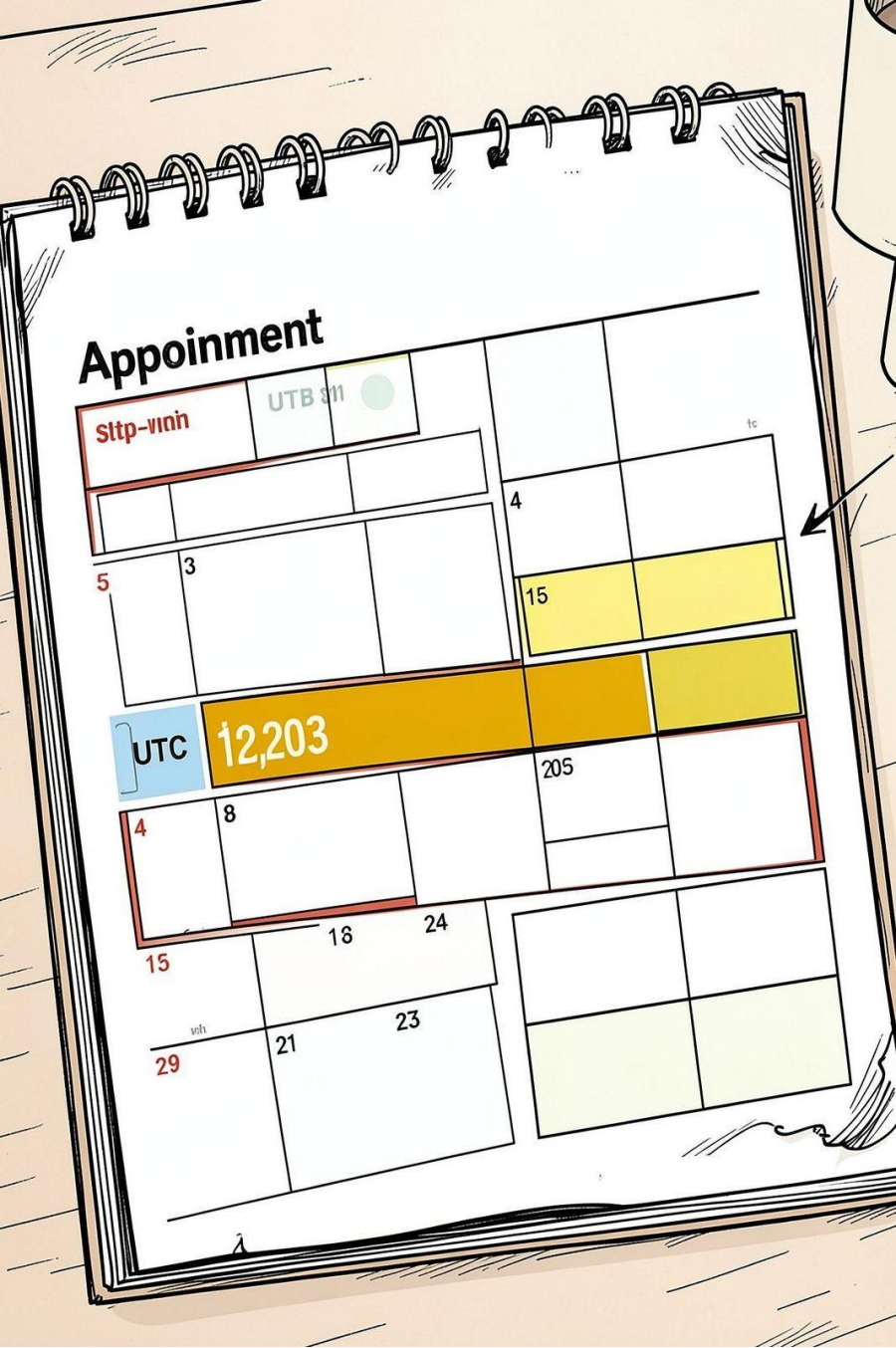
Technical core

TypeScript backend, React SPA, MongoDB replica set, shared types module.



Key requirement

Concurrency safety: database-enforced no-double-booking invariant with tests and transactions.



Problem & Objectives

O1

Problem

Coordinating shared doctor availability: check-then-write is unsafe under concurrent requests.

O\

Primary Objectives

Role-based authenticated access; discover doctors; book appointments; manage holidays; audit sensitive reads; timezone-consistent slots; automated tests with concurrency evidence.

O3

Core Invariant

At most one Appointment per {doctor, startsAt} with status = "booked" enforced by a partial unique index.

Functional Features



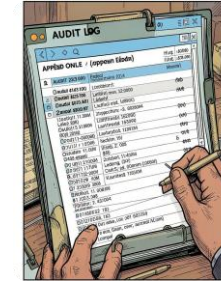
Appointment booking
Month/day views and secure booking flow.



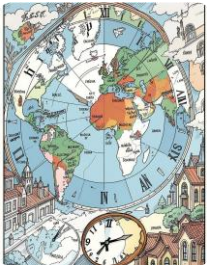
Concurrency-safe booking
Partial unique compound index + transactions; duplicate conflicts map to HTTP 409.



Role-based access
Patient, doctor, receptionist, admin with ownership guards.



Audit & holidays
Append-only audit trail for sensitive reads; clinic-wide and doctor-specific holidays.



Timezone-aware scheduling

Working hours in clinic timezone
→ UTC instants for startsAt/endsAt; viewer display in local timezone.

Technology Stack



Backend

TypeScript 5, Express 4, Mongoose 8, MongoDB replica set



Frontend

React 18, Vite 5, react-router v6, hand-written responsive CSS



Security

bcrypt, JWT access + HttpOnly refresh cookies, refresh-token rotation, rate limiting



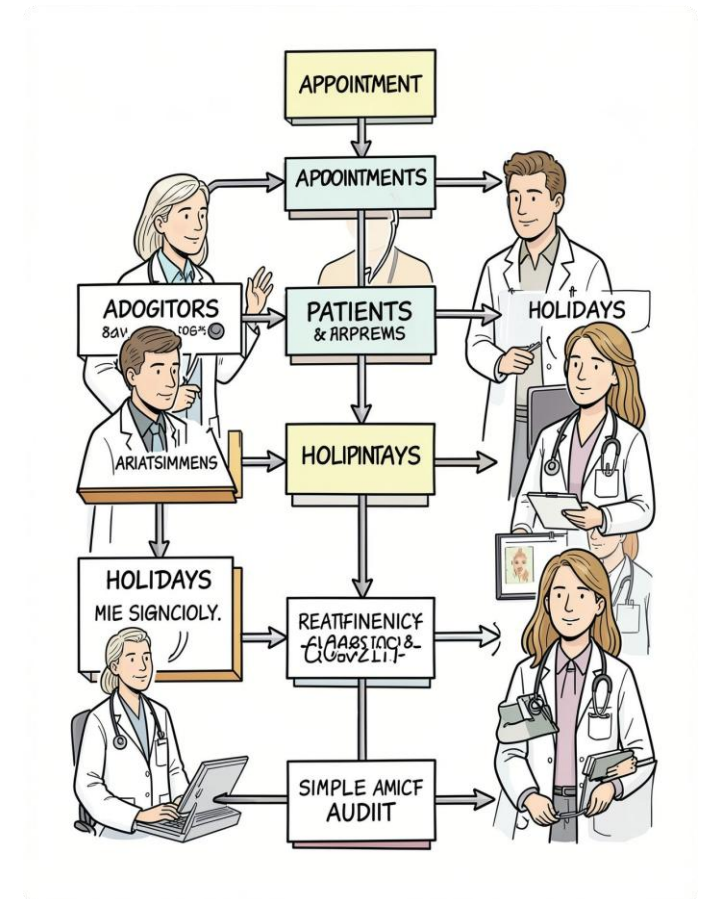
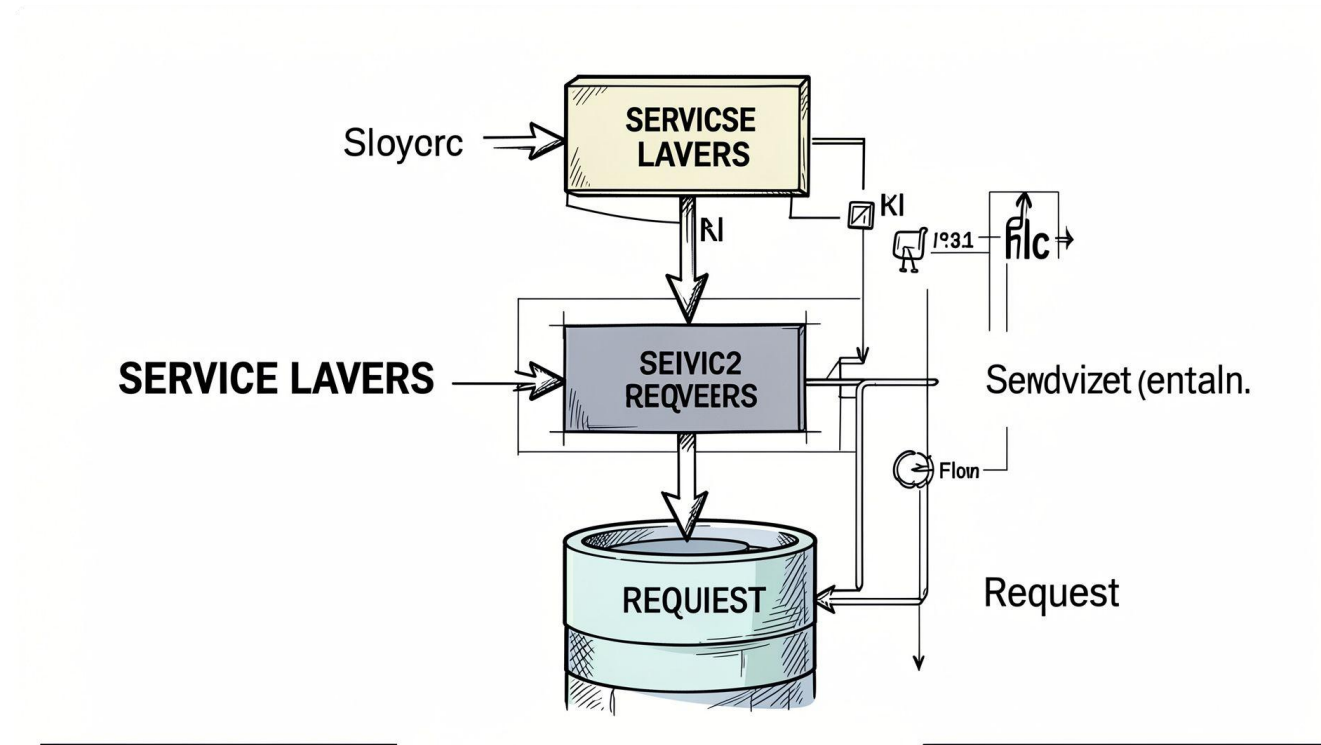
Testing

Jest, supertest, mongodb-memory-server; frontend tests with Node test runner



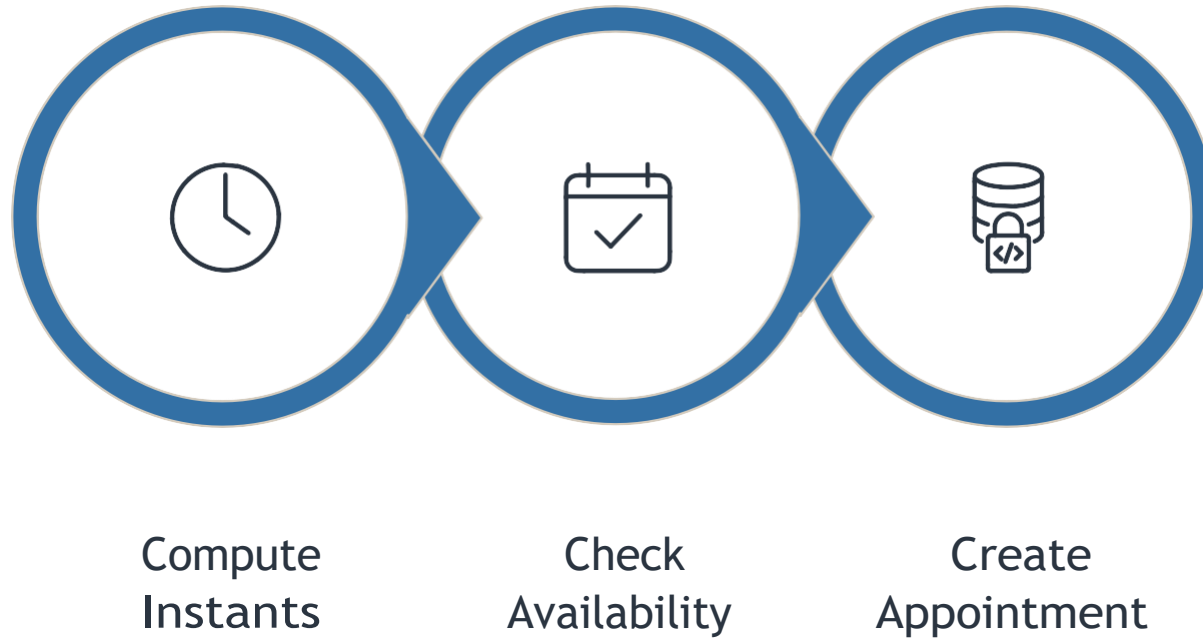
System Architecture & Data Model

Four-layer backend: Routes → Controllers → Services → Models. Frontend: pages, components, auth, api client. Shared types package aligns contracts.



Doctor workingHours embedded; Appointments reference doctor & patient. Partial index frees cancelled slots.

Scheduling, Concurrency & Evidence



Slots computed on demand from working hours (CLINIC_TZ) → converted to UTC instants. Partial unique index on {doctor, startsAt} where status='booked' plus transactions provide database backstop. Concurrency tests: 50 concurrent requests → 1×201, 49×409.

Authentication, Authorization & Security



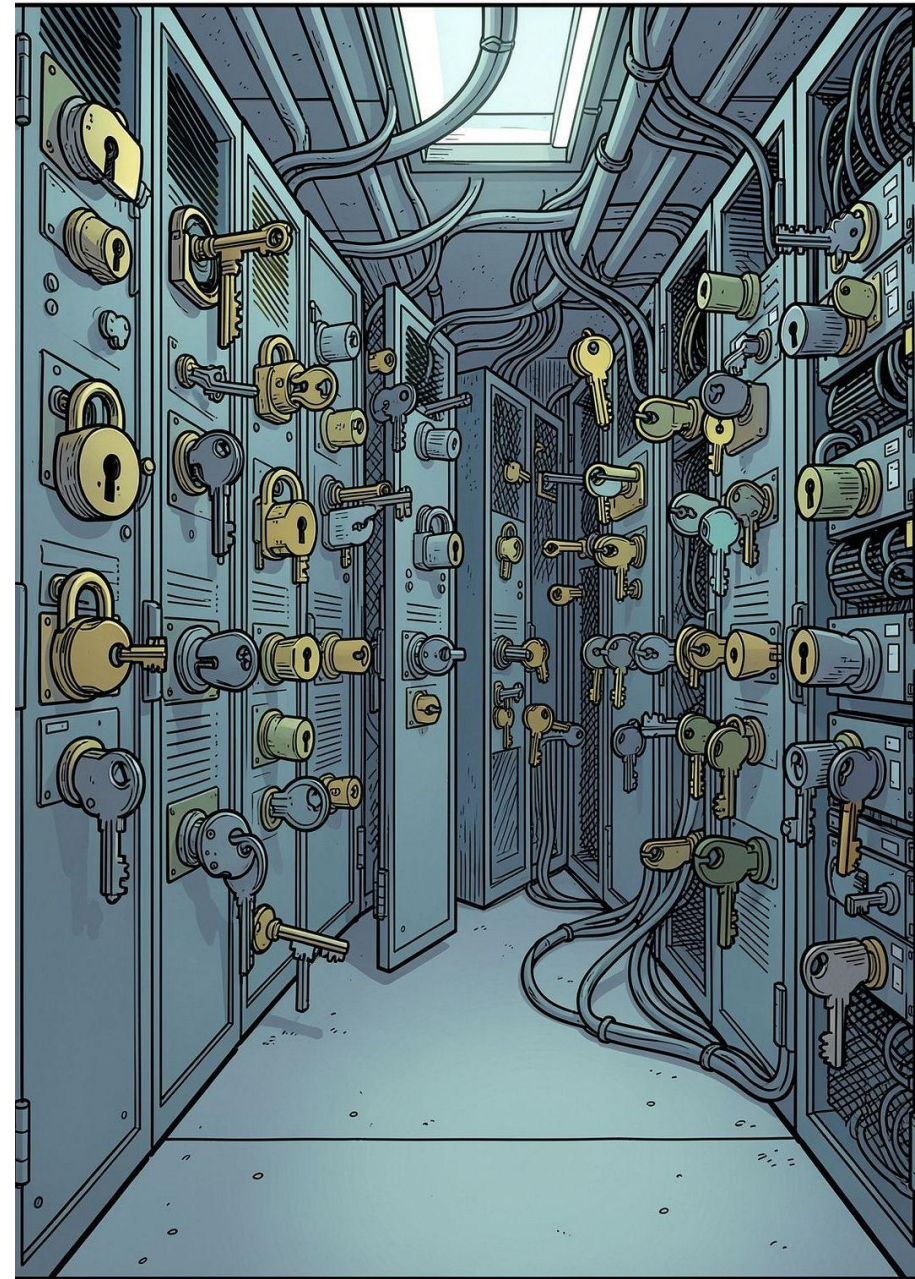
Auth
model
Short-lived
JWT access
tokens;
HttpOnly refresh
cookies;
separate
signing



Authorization
Role guards and
ownership checks;
401 vs 403
distinction
preserved; IDOR
protections.



Operational
controls
Rate limiting
(API/login/booking)
, input validation,
X-Request-Id
tracing,
redaction of
sensitive fields,
CORS with single



API, Frontend & Ops

API highlights

/api root at `http://localhost:4000`; consistent error JSON `{error,message,details}`; key endpoints: `/auth`, `/doctors`, `/patients`, `/appointments`, `/holidays`, `/audit`, `/health`. Standard status codes include 201 for booking and 409 for booking conflicts.

Frontend

React SPA keeps API calls in `api/client`; `AuthContext` and `ProtectedRoute` manage flow; components include `SlotGrid`, `BookingCalendar`, `Toast`.



Installation & Tests

Prereqs: Node 20+, npm 9+, MongoDB replica set. Install backend/frontend, seed demo data, run npm test suites. Concurrency proof and naive comparison available via npm scripts.



Primary invariant and concurrency evidence are documented and exercised by automated tests.

THANK YOU